

Unsupervised Code Representation Learning via Contrastive Learning for Cross-Project Defect Prediction

Hanlin Zhao[1(✉)], Zhiyong Feng[1], and Ruitao Feng[2]

[1] Tianjin University, Tianjin 300072, TJ, China
zha0h4nl1n@tju.edu.cn
[2] Southern Cross University, Gold Coast, Bilinga QLD 4225, Australia

Abstract. Cross-project defect prediction (CPDP) is an approach for addressing the situation that historical data is a few for software defect prediction. Many deep learning algorithms applied for CPDP suffer from the huge demand for labeled data to generated feature, which leads to poor performance in CPDP tasks. To address this situation, we propose the method DPCL generating feature without labels, which is aligned feature by comparative learning. During extracting feature, the cascaded architecture of convolutional neural networks and self-attention mechanism is applied for captured the local information and global dependencies. We also design the framework DPCL based on DPCL for CPDP. Experimental studies on the datasets across 17 different software projects show that DPCL achieved an improvement of up to 28.8% in the F1-score on CPDP. We chose the 17 java projects to evaluate our method compared to the baselines.

Keywords: Contrastive learning · Cross-project defect prediction · Deep learning · CNN · Self-Attention mechanism

1 Introduction

As software systems grow in scale and complexity, identifying software defects through testing has become increasingly challenging and costly. Billions of dollars are spent annually on software testing [1]. Software defect prediction(SDP) is an application for constructing models that identify defect-prone modules early in the software development cycle, enabling developers to allocate limited test resources and arrange the test process effectively [2–5].

Most existing approaches rely on historical defect data from within project to train a predictive model. However, many organizations lack a comprehensive and well-maintained repository of defect records. To address this issue, engineers have turned to (CPDP) [6], which leverages data from other projects to train model [7].

Early CPDP methods relied on hand-crafted static code metrics [8–12] and later on ML algorithms like SVM and LR [13–15]. However, most defects are semantic [16], and these methods fail to exploit the rich semantic/syntactic information in source code and ASTs, limiting predictive power. Recent advances in deep learning have enabled more

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2027
D.-S. Huang et al. (Eds.): ICIC 2026, LNCS 16656, pp. 3–14, 2027.
https://doi.org/10.1007/978-981-92-3441-7_1

semantic feature extraction directly from code representations [3, 17–20]. However, two challenges persist:

Challenge 1: Relying on a Large Amount of Labeled Data for Training. In the real-world software development, modules with defect labels are just a few in the defect database. However, existing deep learning-based research, including DPCNN [19], DPSAM [20], DeeplineDP [21] and LineDef [22], exhibits a substantial demand for labeled data during training.

Challenge 2: Suboptimal CPDP Performance. Existing methods exhibit critical limitations: DP-CNN captures local AST patterns but fails to model global dependencies; DPSAM uses self-attention yet still underperforms; DeepLineDP and LineDef leverage multi-granularity reasoning but remain unsatisfactory. Even LLMs are hindered by input length constraints, risking omission of defect-relevant code at the end of a program [23].

Contrastive learning has achieved notable success in computer vision by learning representations without labels [24]. However, applying it to NLP via data augmentation risks removing defect-related information [25]. We observe that source code offers two naturally aligned views—code token sequences and AST node sequences—which share semantic overlap and can serve as mutual augmentations. Aligning these views through contrastive learning preserves defect-sensitive in an unsupervised manner.

(1). The framework based on the method generated features without labels for CPDP. In this article, we design DPCL, a framework based on contrastive learning to generate code features to adapt to the situation that modules with defect labels are just a few in the defect database. **(2). The cascaded architecture of convolutional neural networks and self-attention mechanism.** Our encoder employs the cascaded architecture of CNNs and self-attention to capture both local code patterns and long-range dependencies. **(3). Align feature by contrastive learning. Features extracted from the code tokens sequence and the AST to kens sequence are aligned with each other to generate feature.** We evaluated the performance of DPCL-SVM on CPDP on 17 java projects from Defects4J. Our framework gets 28.8% highest improvement in F1-score on CPDP.

In a word, our main contributions are as follows:

- We propose a method DPCL, an unsupervised contrastive learning method to generate code representations without labeled data.
- We design DPCL, an end-to-end framework for CPDP which can be adapted to the situation that modules with defect labels are just a few in the defect database.
- We evaluated framework DPCL on 17 Java projects from the Defects4J. Experimental results show that DPCL achieves improvements of up to 28.8% in F1-score over baselines, while also reducing computational overhead.

The remaining part of this article is as follows. Section 2 details our framework DPCL. Section 3 mainly presents our experimental settings, results and analysis. Section 4 reviews related work. Finally, we conclude this work in Sect. 5.

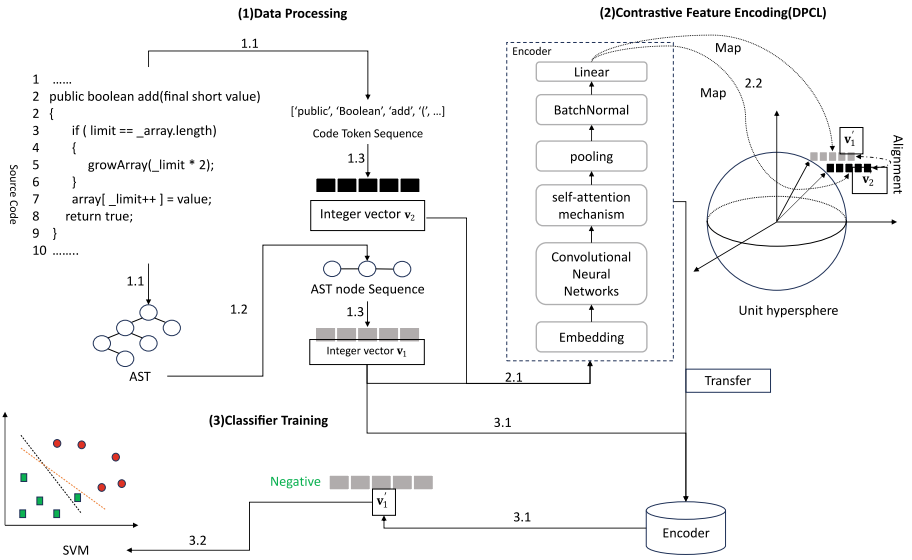


Fig. 1. The overview of DPCL-SVM

2 The Framework of DPCL-SVM

In this section, we will introduce the framework of DPCL-SVM, which builds upon the DPCL method to automatically generate features from the source code through contrastive learning. Figure 2 presents the overview of our framework. The process consists of three main stages:

1. Data processing: Source code is converted into numerical vectors. **1.1.** The source codes are parsed to code tokens sequence through lexical analysis. ASTs are extracted from the source code by syntax analysis. **1.2.** ASTs are traversed using depth first search to extract AST node sequence. **1.3.** Two types of sequence are mapped to integer vectors with separate vocabularies. Vocabulary size is constrained.
2. Contrastive feature encoding (DPCL): In this step, encoders are trained in the way of contrastive learning. **2.1.** Features for two types of sequence are processed by separate but identically structured encoders. The structure of encoder is shown in Fig. 2. Each encoder consists of an embedding layer; CNN and self-attention layers; an average pooling layer for dimensionality reduction and robustness; and a batch normalization layer. **2.2.** Features generated from two types of sequence are aligned within a shared latent space. The alignment ensures the unified representation retains the information common to both syntactic and lexical views of the code.
3. Classifier training: SVM is trained by features produced by the DPCL encoder. **3.1.** Programs are only parsed to ASTs, converted to AST node sequences, and mapped to integer vectors according to step 1. Features are generated from integer vectors by the DPCL encoder. **3.2.** Software defect datasets are typically imbalanced. To deal with the problem of data imbalance, we apply the ADSYN algorithm [26] to generate

pseudo-samples for the minority class. The resulting balanced feature set was and used to train SVM.

Table 1. Selected AST nodes type

Node Category	Node Types
Method invocations and class instances creations	Method Invocation, Super Method Invocation, Class Creator
Declaration nodes	Class Declaration, Package Declaration, Constructor Declaration, Field Declaration, Method Declaration, Variable Declaration, Interface Declaration, Formal Parameter.
Control-flow nodes	If Statement, For Statement, While Statement, Synchronized Statement, Switch Statement, Break Statement, Continue Statement, Return Statement, Throw Statement, Try Statement, Block Statement, Catch Clause Parameter, Try Resource, Catch Clause, Switch Statement, For Control, Enhanced For Control, Do Statement, Assert Statement, Try Statement.

2.1 Data Processing

According to the previous research results [19, 20], not all nodes in the AST contribute equally to identifying software defects. Consequently, we selectively retain nodes, which are shown in Table 1:

- Method invocations and class instances creations, which we record as their method names or class names.
- Declaration nodes, such as method declarations, type declarations, and enumeration declarations, which we record as their node types.
- Control-flow nodes, we also record their node types.

2.2 Contrastive Feature Encoding (DPCL)

In this step, a deep learning encoder is employed to generate feature vectors from the code token sequence and the AST node sequence. Contrastive learning is applied to align the representations, making the encoder to preserve the shared semantic information present in both views of the same program.

The Cascaded Architecture of Convolutional Neural Networks and Self-Attention Mechanism: The encoder's core component is the cascaded architecture of CNN and self-attention mechanism. First, a one-dimensional convolutional layer processes the input sequence in the direction of the sequence. This is followed by an adaptive average

pooling layer. This step reduces the sequence length and decreases the model's computational memory requirements during training. The compressed sequence is then passed to Multi-Head Self-Attention with residual connections, formally defined by Eq. (1):

$$MultiHeadAttention = Concat(h_1, h_2, \cdots, h_n)W^O \tag{1}$$

$$h = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2}$$

Align Feature by Contrastive Learning: Contrastive learning operates on the principle of minimizing the distance between representations of samples from the same category within a learned embedding space We leverage this approach to align the feature representations derived from a program's code token sequence and AST node sequence. By applying a contrastive loss, we explicitly optimize the encoders to map these dual views of the program to proximate vectors in the latent space.

Contrastive Feature Encoding Loss: To align feature, we use cosine similarity. Our loss function is as follows:

$$Loss = \frac{1}{N}\sum_{i=1}^{N} 1 - \frac{x_i^T y_i}{[\![x_i]\!]_2 [\![y_i]\!]_2} \tag{3}$$

x_i and y_i represent generated feature from code token sequence and AST node sequence respectively. N represents the batch size.

The principle of alignment is shown in Fig. 1. The features generated from two type of sequences respectively are mapped to the unit hypersphere in a shared space. Minimizing the contrastive loss directly optimizes this projection, compelling the feature representations of the two modalities to converge, maximizing their similarity.

3 Evaluation

3.1 Research Questions

Our experiment is guided by the following five research questions (RQs), designed to comprehensively evaluate the proposed DPCL framework:

1. RQ1. How does DPCL perform compared to the baseline?
2. RQ2. What is the contribution of the contrastive learning to the overall performance of DPCL?
3. RQ3. How does the cascaded architecture of convolutional neural networks and self-attention mechanism contribute to DPCL?

3.2 Experimental Setup

Dataset

Defects4J [27] is a benchmark dataset for software defect repair, which comprises 17 Java projects. It provides detailed information on defects, including the buggy and fixed

Table 2. The details of dataset based on Defects4j

Project	Buggy Files	All Files	Buggy Rate	Project	Buggy Files	All Files	Buggy Rate
Chart	28	557	5.0%	JacksonDatabind	157	481	32.6%
Cli	51	62	82.3%	JacksonXml	6	39	15.4%
Closure	225	666	33.8%	JxPath	35	181	19.3%
Codec	24	73	32.9%	Jsoup	125	157	79.6%
Collections	28	333	8.4%	Lang	61	114	53.5%
Compress	58	231	25.1%	Math	117	224	52.2%
Csv	17	22	77.3%	Mockito	46	199	23.1%
Gson	21	76	27.6%	Time	31	168	18.5%
JacksonCore	35	128	27.3%	Total	1,065	3,711	28.7%

versions, types of defects. The projects in Defects4j include not only buggy files but also files unrelated to the defects. Therefore, we construct dataset based on defective files and files unrelated to the defects for SDP. Details are shown in Table 2.

Implementation

For the model parameters, the size of vocabulary is 5,000. The embedding dimension is set as 24. The CNN layer uses a filter length of 8. The sequence is compressed to a fixed length of 5,000 elements. In multi-head attention, the number of head is 1, the number of hidden nodes is 24. The parameters of SVM are set to default configurations suggested by Scikit-learn. The learning rate is set to 0.001.

Metrics

In this article, we use four metrics to evaluate our framework, including accuracy, precision, recall and F1 score. These metrics are widely used in machine learning and deep learning [3, 19, 20]. The calculation formulas of four metrics are as follows. TP is the counts of actually buggy files predicted as buggy, and correspondingly, FP is the counts of actually clean files predicted as buggy. FN is the counts of actually buggy files predicted as clean, and TN is the counts of actually clean files predicted as clean.

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN} \tag{4}$$

$$Precision = \frac{TP}{TP + FP} \tag{5}$$

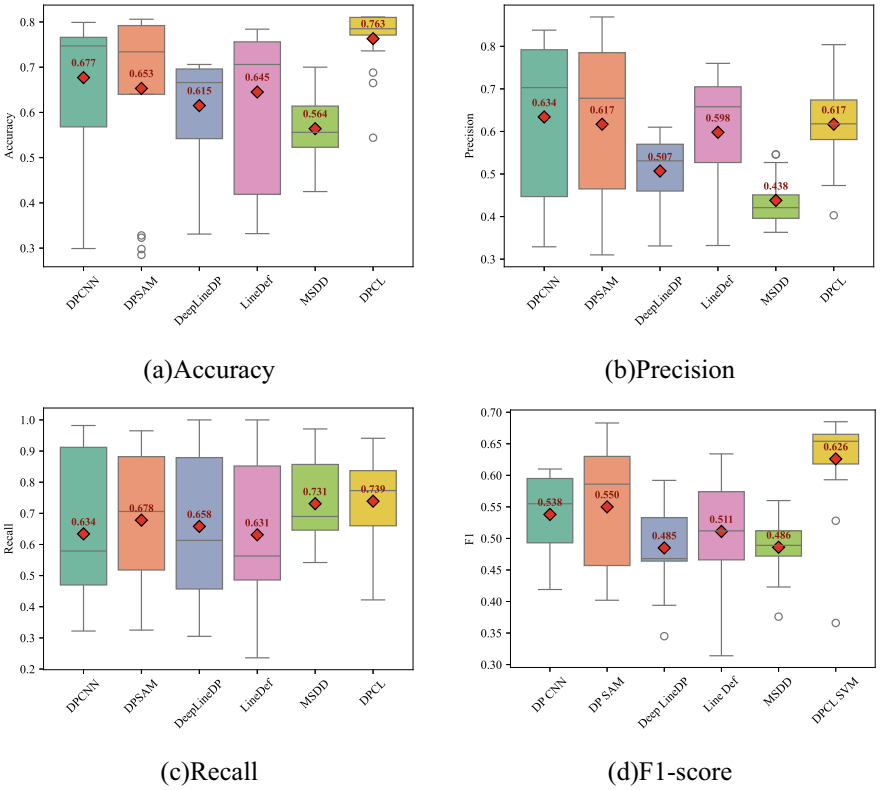
$$Recall = \frac{TP}{TP + FN} \tag{6}$$

$$F1 - score = \frac{2 \times Precison \times Recall}{Precision + Recall} \tag{7}$$

Baseline Methods
The baseline methods related to RQ1 are listed as follows:

- DPCNN [19]: CNN + static metrics → logistic regression.
- DPSAM [20]: Self-attention + static metrics → logistic regression.
- DeepLineDP [22]: BiGRU-based, models token/line contexts for file & line-level prediction.
- LineDef [23]: GCN-based, joint reasoning across code granularities for file & line-level prediction.
- MSDD [24]: CodeT5+ LLM fine-tuned with $(IA)^3$ for defect prediction.

3.3 RQ1. How Does DPCL-SVM Perform Compared to the Baseline?



(a)Accuracy (b)Precision

(c)Recall (d)F1-score

Fig. 2. The Experimental result of CPDP

In experiments, we compared our proposed method with baselines on the CPDP task. We conduct a series of experiments where each of the 17 projects serves as the training set in turn, with the remaining 16 projects used for testing. The metrics reported for each test project are averaged to produce the final result for that training set. The experiment results are presented in Fig. 2.

Our analysis primarily focuses on the F1-score. DPCL achieves an average F1-score of 0.626. This represents a significant improvement over the baseline methods: 21.6% higher than DPCNN, 13.4% higher than DPSAM, 28.8% higher than MSDD, 29.1% higher than DeepLineDP, and 22.6% higher than LineDef.

Regarding other metrics, DPCL does not achieve the highest average precision, which is superior in DPCNN and DPSAM. However, it attains the highest average recall among all methods, showing the highest 23.6% improvement. Compared to MSDD, DPCL shows superior performance in both precision and recall for the CPDP task. The suboptimal performance of MSDD may be attributed to the input length limitation of LLMs. Additionally, DPCL shows the highest average accuracy compared to baselines.

Answer to RQ1: The experiment result demonstrates that DPCL outperforms all baseline methods. It also shows ability to distinguish with defective files, which is beneficial to reduce potential risks and cost caused by defects.

3.4 RQ2. How Does DPCL-SVM Perform Compared to the Model Without Contrastive Learning?

To evaluate the contribution of contrastive learning, we conduct an ablation study. Therefore, we design a variant of our model. The details are shown as follow:

- noCL: a classification head is attached directly to the output of the cascaded CNN and self-attention encoder in Fig. 1, which is then trained with labels.

We evaluate the model on the CPDP task. The results are displayed in Table 3.

Table 3. The Experimental results on CPDP of Ablation Study on contrastive learning

Project	Accuracy		Precision		Recall		F1	
	noCL	DPCL	noCL	DPCL	noCL	DPCL	noCL	DPCL
Avg	0.728	0.763	0.581	0.617	0.720	0.739	0.579	0.626

We observe that DPCL outperforms noCL in CPDP across all metrics: accuracy, precision, recall, and f1-score. DPCL achieves an 8.1% higher average F1-score over noCL. It also obtains 4.8% higher than noCL on accuracy, gets 6.2% improvement over noCL on precision and show slightly improvement over noCL on recall.

Answer to RQ2: The ablation study confirms that the contrastive learning is highly effective. It consistently improves the predictive performance of the model, validating its essential role within the DPCL framework for cross-project defect prediction.

3.5 RQ3. How Does the Cascaded Architecture of Convolutional Neural Networks and Self-Attention Mechanism Affect DPCL-SVM?

To assess the contribution of CNN within the cascaded architecture, we conduct a second ablation study. We construct a variant, which is shown as follow:

- CLSAM: CLSAM is removed CNN layer from the encoder shown in Fig. 1.

Our evaluation focused on two critical aspects of computational efficiency: the maximum allocated GPU memory and the training time per epoch. These measurements were recorded under a consistent experimental setup using a batch size of 4. The result is shown in Tables 4 and 5.

The results reveal significant computational advantages for DPCL. As shown in Table 4. DPCL requires approximately one-eighth of the GPU memory of CLSAM. This substantial reduction in the GPU memory is a direct benefit of the initial convolutional layer, which compresses the sequence length before the self-attention mechanism. Furthermore, DPCL completes each training epoch nearly 17 times faster than CLSAM.

Table 4. The Experimental result of Ablation Study on Training Cost

	The max allocated GPU memory	Time cost
CLSAM	19,718.01 MB	4.03 min
DPCL	2,429.40 MB	0.23 min

Regarding predictive performance on the CPDP task, Table 5 shows that both models achieve comparable average F1-scores. We observe a trade-off: while CLSAM attains a higher average recall, DPCL achieves superior average accuracy and precision.

Answer to RQ3: The experimental results indicate the cascaded architecture of convolutional neural networks and self-attention mechanism re duces the GPU memory and time cost used in training process while maintaining the model's performance comparable to a pure self-attention model (CLSAM).

Table 5. The Experimental result of Ablation Study on CPDP

Project	Accuracy		Precision		Recall		F1	
	CL SAM	DPCL	CL SAM	DPCL	CL SAM	DPCL	CL SAM	DPCL
Avg	0.734	0.763	0.597	0.617	0.768	0.739	0.612	0.626

4 Related Work

4.1 Program Analysis in CPDP

SDP is a process of predicting whether the code module to cause defects. The general workflow for SDP in volves three key steps: (1) collect and preprocess historical project data, (2) extract features and train a classification model, (3) evaluate the model's performance on the test set.

In the early days, researchers tried to do program analysis of the source code. Zimmermann [28] applied network analysis techniques on program dependency graphs to identify program that are more prone to defects. Additionally, researchers focused on designing static code metrics. These metrics include lines of code (LOC) [9] and Halstead's complexity measures [10], object-oriented suites like CK and MOOD [11, 12]. However, they are fundamentally limited: they do not encode the rich semantic information in the source code.

4.2 Machine Learning in CPDP

With the advancement of machine learning, researchers began to use these static metrics as code features to train classifiers of machine learning for defect prediction. This shift enabled more automated and data-driven prediction models. Dejaeger [29] developed interpretable software fault prediction models using Bayesian network classifiers, while Cruz [13] proposed a method that enhances the capability of logistic regression models in cross-project prediction of defect-prone code through data transformation.

4.3 Deep Learning in CPDP

The rapid development of deep learning enabled a paradigm shift: learning feature representations directly from the source code. Wang et al. first used deep belief network (DBN) to automatically learn semantic information from the abstract syntax tree (AST) of the program [18]. Subsequent work refined this direction. Li applied Convolutional Neural Networks to AST node sequences [19]. Liang [30] et al. employed the LSTM to extract semantic information from selected AST nodes.

5 Conclusion

In this paper, we design DPCL, a framework based on contrastive learning to generate features without requiring defect labels for CPDP. The framework employs a cascaded architecture of CNN and self-attention layers within its encoder to comprehensively capture both local patterns and global dependencies from code token and AST node sequences. By aligning these dual representations via contrastive loss, DPCL effectively learns a unified feature space in a self-supervised manner. This approach is particularly valuable for the real scenario where defect databases contain only a limited number of labeled modules. The experimental evaluation on 17 projects from the Defects4J benchmark demonstrates DPCL achieves state-of-the-art performance, obtaining F1-score improvements ranging from 13.4% to 28.8% compared to baselines.

Acknowledgments. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Xu, J., Chen, P., Yin, B., Huang, Z., Qiu, Q.: FINDGATE: fine-grained defect prediction based on a heterogeneous discrete code graph-guided attention transformer. In: IEEE 23rd International Conference on Software Quality Reliability, and Security (QRS), pp. 72–82. IEEE, Piscataway, NJ (2023)
2. Chen, X., Zhang, D., Zhao, Y., Cui, Z., Ni, C.: Software defect number prediction: unsupervised vs supervised methods. Inform. Softw. Technol. **106**, 161–181 (2019)
3. Zhang, X., Ben, K., Zeng, J.: Cross-entropy: a new metric for software defect prediction. In: 2018 IEEE International Conference on Software Quality Reliability and Security, pp. 111–122. IEEE, Piscataway, NJ (2018)
4. Hall, T., Beecham, S., Bowes, D., Gray, D., Counsell, S.: A systematic literature review on fault prediction performance in software engineering. IEEE Trans. Softw. Eng. **38**(6), 1276–1304 (2012)
5. Chen, H., Jing, X., Li, Z., Wu, D., Peng, Y., Huang, Z.: An empirical study on heterogeneous defect prediction approaches. IEEE Trans. Softw. Eng. **47**(12), 2803–2822 (2020)
6. Ahmed, I. et al.: Artificial intelligence for software engineering: the journey so far and the road ahead. ACM Trans. Softw. Eng. Methodol. **34**(5), 1–27 (2025)
7. Khatri, Y., Singh, S.K.: Cross project defect prediction: a comprehensive survey with its SWOT analysis. Innov. Syst. Softw. Eng. **18**(2), 263–281 (2022)
8. Basili, V.R., Briand, L.C., Melo, W.L.: A validation of object-oriented design metrics as quality indicators. IEEE Trans. Softw. Eng. **25**, 456–473 (1996)
9. Akiyama, F.: An example of software system debugging. Int. J. Perform. Eng. **71**, 353–359 (1971)
10. Halstead, M.: Elements of Software Science (Operating and Programming Systems Series). Elsevier Science Inc., New York (1977)
11. Chidamber, S., Kemerer, C.: A metrics suite for object-oriented design. IEEE Trans. Softw. Eng. **20**(6), 476–493 (1994)
12. Harrison, R., Counsell, S., Nithi, R.: An evaluation of the MOOD set of object-oriented software metrics. IEEE Trans. Softw. Eng. **24**(6), 491–496 (1998)
13. Cruz, A.E.C., Ochimizu, K.: Towards logistic regression models for predicting fault-prone code across software projects. In: 2009 3rd International Symposium on Empirical Software Engineering and Measurement, pp. 460–463. IEEE Press, New York (2009)
14. Shan, C., Chen, B., Hu, C., Xue, J., Li, N.: Software defect prediction model based on LLE and SVM. In: 2014 Communications Security Conference (CSC 2014), pp. 1–5. IEEE, New York (2014)
15. Meinke, K., Bennaceur, A.: Machine learning for software engineering: models, methods, and applications. In: 2018 ACM/IEEE 40th International Conference on Software Engineering: Companion (ICSE-Companion), pp. 548–549. IEEE, Piscataway, NJ (2018)
16. Li, Z., Tan, L., Wang, X., Lu, S., Zhou, Y., Zhai, C.: Have things changed now? An empirical study of bug characteristics in modern open source software. In: ASID '06: Proceedings of the 1st Workshop on Architectural and System Support for Improving Software Dependability, pp. 25–33. ACM, New York (2006)
17. White, M., Vendome, C., Linares-Vásquez, M., Poshyvanyk, D.: Toward deep learning software repositories. In: Proceedings of the 12th Working Conference on Mining Software Repositories, pp. 334–345. IEEE Press, Piscataway (2015)
18. Wang, S., Liu, T., Tan, L.: Automatically learning semantic features for defect prediction. In: Proceedings of the 38th International Conference on Software Engineering, ICSE 2016, Austin, TX, USA, May 14–22, 2016, pp. 297–308. ACM, New York (2016)

19. Li, J., He, P., Zhu, J., Lyu, M.R.: Software defect prediction via convolutional neural network. In: 2017 IEEE International Conference on Software Quality Reliability and Security (QRS), July 25–29, 2017, pp. 318–328. IEEE Computer Society, Los Alamitos, California (2017)
20. Yu, T.Y., Huang, C.Y., Fang, N.C.: Use of deep learning model with attention mechanism for software fault prediction. In: Proceedings of 2021 8th International Conference on Dependable Systems and Their Applications (DSA) 5–6, 2021, pp. 161–171. IEEE, Piscataway (2021)
21. Pornprasit, C., Tantithamthavorn, C.K.: DeepLineDP: towards a deep learning approach for line-level defect prediction. IEEE Trans. Softw. Eng. **49**(1), 84–98 (2023)
22. Yin, S. et al.: Line-level defect prediction by capturing code contexts with graph convolutional networks. IEEE Trans. Softw. Eng. **51**(1), 172–191 (2025)
23. Wang, X., Lu, L., Yang, Z., Tian, Q., Lin, H.: Parameter-efficient multi-classification software defect detection method based on pre-trained LLMs. Int. J. Comput. Intell. Syst. **17**, 152 (2024). https://doi.org/10.1007/s44196-024-00551-3
24. Misra, I., van der Maaten, L.: Self-supervised learning of pretext-invariant representations. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, June 14–19, 2020, pp. 6706–6716. IEEE, Piscataway (2020)
25. Rethmeier, N., Augenstein, I.: A primer on contrastive pretraining in language processing: methods, lessons learned, and perspectives. ACM Comput. Surv. **55**(10), 1–17 (2023)
26. He, H., Bai, Y., Garcia, E.A., Li, S.: ADASYN: adaptive synthetic sampling approach for imbalanced learning. In: Proceedings of the 2008 IEEE International Joint Conference on Neural Networks (IJCNN 2008), Hong Kong, China, June 1–8, 2008, pp. 1322–1328. IEEE, Piscataway (2008)
27. Just, R., Jalali, D., Ernst, M.D.: Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In: Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA 2014), San Jose, CA, USA, July 21–25, 2014, pp. 437–440. ACM, New York (2014)
28. Zimmermann, T., Nagappan, N.: Predicting defects using network analysis on dependency graphs. In: Schäfer, W., Dwyer, M.B., Gruhn, V. (eds.) Proceedings of the 30th International Conference on Software Engineering, pp. 531–540. ACM, New York (2008)
29. Dejaeger, K., Verbraken, T., Baesens, B.: Toward comprehensible software fault prediction models using Bayesian network classifiers. IEEE Trans. Softw. Eng. **39**(2), 237–257 (2012)
30. Alhija, H.A., Azzeh, M., Almasalha, F.: Predicting defect-prone software modules using support vector machines. J. Syst. Softw. **81**(5), 649–660 (2008)